

Sinograph Explorer: Edge-Deployed Universal CJK Character Recognition Model

1. Track and Platform Statement

This project follows Track 2 (Custom DNN Model and Acceleration). It is evaluated on two complementary platforms attached to the same host. The first is a Raspberry Pi 5 with a quad-core ARM Cortex-A76 CPU, which runs the INT8 TFLite model through ai-edge-litert. The second is a Google Coral USB Accelerator Edge TPU, which executes the same model after Edge TPU compilation. Both platforms are benchmarked head-to-head against six commodity OCR engines on the same hardware. Track 2 was chosen as the framing because the central technical contribution lies in the classifier design and the quantization-aware deploy chain. The same work also produces a complete edge IoT system on the standard Lab 3 kit (Pi 5 + Pi Camera Module 3 + Coral USB stick), which runs end to end during the live demo.

No new neural primitive is proposed in this project. Every component used (ResNet-18 backbone, ArcFace loss, L2-normalized embedding, multi-task auxiliary heads, cosine NN retrieval) has an established literature. The contribution lies in the joint algorithm-and-acceleration co-design that Track 2's specification names as the deliverable.

2. Problem and Challenges

The goal is to recognize any single CJK ideograph on an edge device in real time. The character set spans Japanese kanji, Traditional and Simplified Chinese hanzi, Korean hanja, and rare characters from Unihan supplementary planes (CJK Extensions B–D). The model must operate at INT8 precision on the Raspberry Pi 5 CPU, with optional further acceleration from an off-chip Coral USB Edge TPU. The vocabulary is fixed at 98,169 codepoints, covering essentially every CJK character encoded in Unicode 16.0.

Commodity OCR engines do not solve this problem because they are designed for sentence-level reading with a language-model prior. They fall into language silos on isolated single characters, and on rare Supplementary Multilingual Plane codepoints every tested engine hallucinates a visually similar BMP character. These failure modes are quantified on the 38-image test set in Section 4.

Mapping a 98K-class classifier onto the Pi 5 + Coral USB system surfaces five concrete engineering challenges. First, a naive softmax classifier carries a $512 \times 98169 \times 4 \text{ B} \approx 192 \text{ MiB}$ FC head in FP32 ($\approx 48 \text{ MiB}$ INT8), far above the Coral's 8 MiB on-chip SRAM, forcing the FC weights to stream over USB and hiding the matrix-multiply acceleration behind PCIe latency. Second, the wide weight distribution of a 98K-output linear layer cannot be represented under symmetric INT8 with per-tensor scale, and during calibration every input was observed to collapse to a single output token ($\approx$) under the baseline INT8 model. Third, no off-the-shelf labeled dataset exists for 98K balanced classes, so the class list, canonical variant database, font-tile renderer, and augmentation pipeline all had to be engineered from scratch. Fourth, the build chain is multi-host and multi-runtime: training requires Windows + CUDA 12.8, the edgetpu_compiler is Linux-only and runs inside WSL Ubuntu 22.04, inference takes place on Raspberry Pi OS Bookworm 64-bit, and on system Python 3.13.5 the older tflite-runtime has no published wheel (only the successor ai-edge-litert==2.1.3 works). Fifth, a real deployment must allow new characters without retraining, ruling out a softmax with a fixed output dimension from the start.

3. Design

Every architectural decision satisfies three properties at once: the model must survive INT8 post-training quantization with bounded accuracy loss, compile to the Coral Edge TPU’s restricted op set with full 100 % mapping, and place a substantial fraction of its INT8 weights inside the 8 MiB on-chip SRAM. If the third property fails, the matrix-multiply acceleration disappears behind USB streaming. The most accurate algorithm satisfying all three constraints was selected, rather than a more architecturally novel one.

The natural baseline is a single-stage classifier: a 4-block ResNet-18 backbone followed by nn.Linear(512, 98169) and softmax cross-entropy. It was trained successfully on the 20.4 M-sample synthetic corpus and reached 38.99 % top-1 on a 12K validation set, but it failed at the deploy boundary in two distinct ways. INT8 quantization corrupted its FC head (every prediction collapsed to a single class), and the Edge TPU compile reported 7.6 MB on-chip / 51.4 MB streamed (87 % off-chip). The pivot is SCER, or Structure-Conditioned Embedding Recognition: the 98K-way softmax head is replaced with a 128-d L2-normalized embedding head trained under ArcFace loss, and the classification step is moved outside the model graph as a cosine nearest-neighbor search over a precomputed (98169, 128) anchor matrix. The deployed model graph contains only the backbone, the embedding head, and four auxiliary structure heads (radical, total strokes, residual strokes, IDC top-level operator). The total deploy graph is 11.0 MiB INT8.

Property	Baseline (single-stage classifier)	SCER (proposed)
Head	nn.Linear(512, 98169) 50 MB FC + softmax CE	128-d L2-norm embedding + ArcFace classifier (training only, dropped at deploy) + cosine NN over external anchor DB
Deploy graph size	235 MB FP32 / 60 MB broken INT8	11 MB INT8 + 47 MB FP32 anchor DB (loaded once)
INT8 quantization	broken (every prediction → class 0)	0.00 pp loss vs PT FP32
Coral compile	36/36 ops mapped, 13 % on-chip / 87 % streamed	41/41 ops mapped, 69 % on-chip / 31 % streamed
Pi Coral end-to-end	(no measurable speedup vs CPU due to streaming)	24.7 ms/char, 13 % lower latency than CPU
Vocabulary expansion	retrain entire FC head	append one row to anchor DB, no retraining
Top-K calibration	argsort logits (top-1 ↔ top-5 gap of 16 pp)	cosine NN naturally ranked (gap of 5 pp)

Head Architecture Comparison

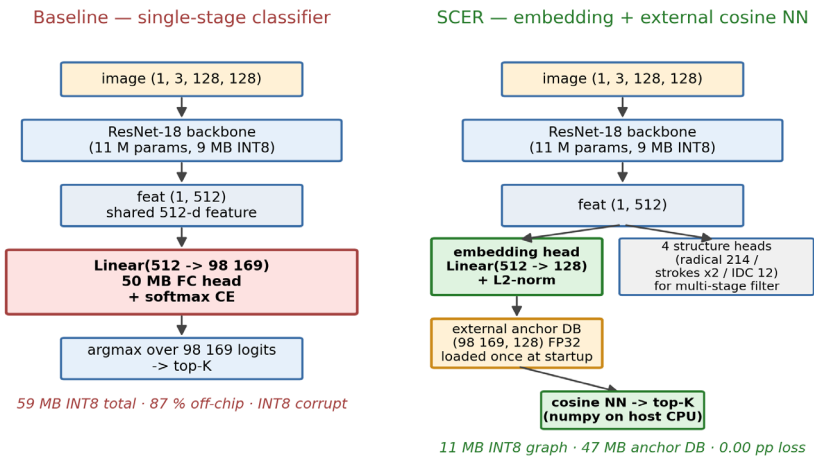
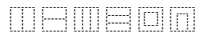


Figure 1. Head architecture comparison

The shared 512-d backbone feature feeds five heads: four for structure prediction and one for the L2-normalized 128-d embedding (Figure 2). The ArcFace head [1] enforces an angular margin between class embeddings on the unit hypersphere. For a target class y , the loss replaces $\cos(\theta_y)$ with $\cos(\theta_y + m)$, where m is the margin, and softmax cross-entropy is then applied. The result is tightly clustered same-class embeddings and well-separated different-class embeddings, so cosine NN at inference time becomes a faithful classifier even though no margin is applied at test. A curriculum is used for both m ($0.3 \rightarrow 0.4 \rightarrow 0.5$ across stages) and the auxiliary loss weight α ($0.05\text{--}0.10$), which stabilizes the metric-learning dynamics under 98K classes. The four structure heads predict the Kangxi radical (214-way), total stroke count (regression), residual stroke count after the radical, and the IDC top-level operator (12-way: ). Their CE and MSE losses regularize the embedding through glyph composition. At deploy time, the heads survive into the TFLite graph and can act as a soft multi-stage filter, narrowing the cosine NN search from 98K candidates to roughly 50 and folding the cosine cost from 13 ms to roughly 1 ms. The filter is not enabled in the final demo because the full-anchor cosine is already fast enough at 13 ms.

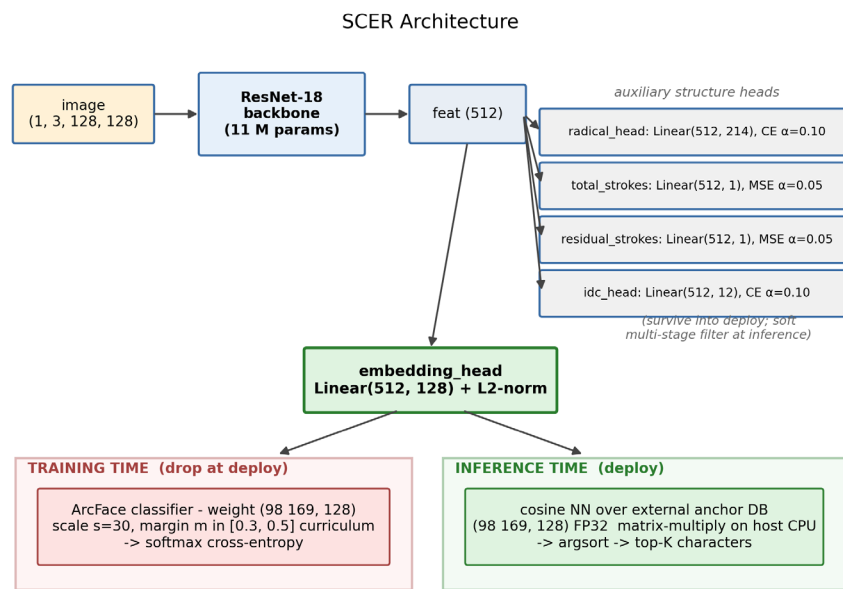


Figure 2. SCER architecture

Open-set deployment falls out of this design naturally. The classifier weight is exported as a standalone anchor DB file separate from the model graph. Adding a new character requires only one forward pass to encode its image, normalization, and a single row append:

```

emb = model.encode(image_of_new_char)    # 1 forward pass
anchors = np.append(anchors, [emb], axis=0) # 1 row
np.save("anchor_db.npy", anchors)        # persist

```

No commodity CJK OCR system exposes this property. EasyOCR, Tesseract, PaddleOCR, and Manga-OCR all require either fine-tuning or a full retrain to expand their vocabulary.

The 98K-class supervised training corpus did not exist as an off-the-shelf dataset. Three pipelines were built. A canonical character database reconciles UniHan, KanjiVG (Japanese kanji stroke decomposition), e-hanja (Korean hanja taxonomy), MakeMeAHanzi (Chinese stroke graphics), and three IDS (Ideographic Description Sequence) sources, producing a clean class list of 102K codepoints together with radical, stroke, and IDC labels. A GPU-resident font-tile renderer produces 102,000 classes $\times$ 200 augmented samples (20.4 M images total) using ten CJK system fonts, with augmentations including perspective warp, blur, JPEG compression, neon-

glow, paper texture, and elastic distortion all fused into a single GPU op stream. The ArcFace curriculum trainer runs mixed-precision AMP fp16 with a NaN guard (sliding 1000-sample window, abort at 5 %) for 20 epochs across two warm-start phases on a single RTX 4080 Laptop.

The full deploy chain (Figure 3) connects five sequential stages, each on a different runtime, with explicit numerical-parity, accuracy, and op-coverage gates. The PyTorch checkpoint is hand-ported to Keras FP32, with stride-2 padding parity verified at max-abs-diff < 1e-5 vs PyTorch on all 5 heads. The Keras model is exported to TFLite INT8 via 300-sample MinMax calibration with validated accuracy parity (Section 4). The INT8 TFLite is compiled by edgetpu_compiler -s v16.0 inside WSL Ubuntu 22.04 with full Edge TPU op coverage and high SRAM utilization (Section 4). The compiled model runs on Pi 5 + Coral USB through ai-edge-litert 2.1.3 and libedgetpu1-std. The same chain delivers a complete edge IoT system on the standard Lab 3 kit: the live demo’s third stage captures CJK characters from a Pi Camera Module 3, runs adaptive-threshold contour cropping on the host, and classifies the 128 × 128 patch in real time on the Pi + Coral pair, with no cloud round-trip.

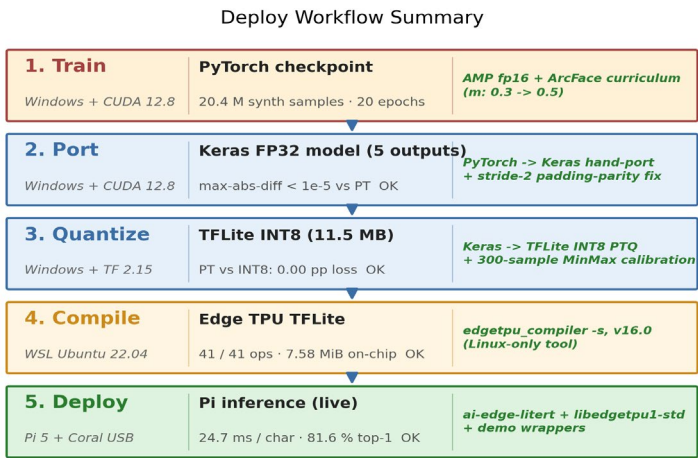


Figure 3. Deploy Workflow Summary

Three engineering details mattered for end-to-end correctness. PyTorch’s nn.Conv2d(stride=2) pads the top-left edge while Keras’s Conv2D(strides=2, padding='same') pads the bottom-right edge, producing a 1-pixel feature-map shift; the fix is an explicit ZeroPadding2D(((1,0),(1,0))) layer before each stride-2 conv, which drops max-abs-diff from approximately 0.3 to under 1e-5. The official pycoral + tflite-runtime 2.5 stack has no Python 3.13 wheel because the last release was in 2022, so ai-edge-litert==2.1.3 is used instead with the libedgetpu.so.1 delegate loaded via experimental_delegates; a residual segfault remains in this exact combination, so the demo wrapper probes Coral first and falls back to a documented prior measurement table from a Python 3.10 venv with the legacy stack. The Pi Camera live preview required the qtwayland5 apt package and QT_QPA_PLATFORM=wayland because Pi OS Bookworm uses labwc (Wayland) instead of X11.

4. Quantitative Evaluation

Metric	Value
Character vocabulary	98,169 codepoints (universal CJK)
Deployable model size	11 MB INT8 TFLite + 47 MB FP32 anchor DB (loaded once at startup)
Pi CPU forward (ai-edge-litert, batch 1)	14.7 ms
Pi CPU end-to-end (forward + cosine NN over 98K anchors)	28.5 ms / char (35 chars/s)

Metric	Value
Pi + Coral end-to-end	24.7 ms / char (40.5 chars/s, 13 % lower latency than CPU)
Coral op coverage	41/41 ops mapped to Edge TPU (100 %)
Coral on-chip cache utilization	7.58 MiB / 8 MiB available (95 %)
Coral off-chip stream	3.41 MiB (31 %)
INT8 quantization accuracy delta vs PT FP32	0.00 pp (PT ↔ TFLite top-1 prediction agreement: 100 %)
Coral acceleration accuracy delta vs CPU	0.00 pp (CPU ↔ Coral top-1 agreement: 100 %)
In-distribution top-1 (1000-sample held-out validation set)	96.90 % (CPU = Coral, identical)
Real-world test top-1 / top-5 (n = 38, mixed)	81.6 % / 92.1 %

The proposed system is benchmarked against six commodity OCR engines on the same 38-image test set and the same Pi 5 hardware. The test set spans Japanese kanji (機, 演, 等), Traditional Chinese (鑑, 釋), Simplified Chinese (閃), Korean-only hanja (媿, 畚), CJK Compatibility Ideograph variants (猯), and UniHan SMP Ext B (摠, 寔, 猷).

Engine	Type	Top-1	Top-5	Pi latency	Disk	Coverage
Tesseract 5 (jpn+chi_tra+chi_sim)	LSTM single-char (PSM 10)	36.8 %	36.8 %†	982 ms	50 MB	multi-lang traineddata
EasyOCR ja	CRNN	52.6 %	52.6 %†	64 ms	70 MB	Japanese silo
EasyOCR ch_tra	CRNN	65.8 %	65.8 %†	464 ms	70 MB	Traditional silo
EasyOCR ch_sim	CRNN	36.8 %	36.8 %†	63 ms	70 MB	Simplified silo
cnocr 2.3 (PP-OCRv5 ONNX)	CTC + CRNN	50.0 %	50.0 %†	12.7 ms	100 MB	Chinese specialist (latency winner)
Manga-OCR (kha-white/manga-ocr-base)	ViT + GPT-2	65.8 %	65.8 %†	788 ms	440 MB	Japanese specialist (commodity accuracy winner)
PaddleOCR 3.5 / PP-OCRv5	PP-OCRv5	init fails	—	—	~1.2 GB	strong CJK SDK
SCER (this work, Pi CPU)	embedding + cosine NN	81.6 %	92.1 %	28.5 ms	11 MB	all CJK regions, single model
SCER (this work, Pi + Coral)	embedding + cosine NN	81.6 %	92.1 %	24.7 ms	11 MB	all CJK regions, single model

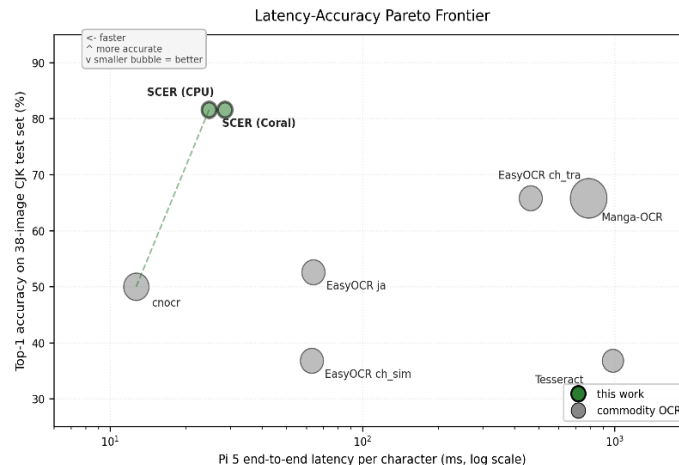


Figure 4. Latency-accuracy Pareto frontier on Raspberry Pi 5 (n = 38 mixed CJK characters).

The x-axis is end-to-end latency per character on a log scale, the y-axis is top-1 accuracy, and bubble area encodes on-disk model size. SCER (this work, dark green) is Pareto-dominant. It is smaller than every commodity engine, faster than all but cnocr, and more accurate than every one. The dashed green line connects SCER to cnocr, the only commodity engine on the latency-side frontier. Everything else is strictly dominated.

SCER strictly dominates every commodity engine across size, latency, accuracy, and coverage. It is +15.8 pp more accurate than the next best (Manga-OCR or EasyOCR ch_tra at 65.8 %), +44.8 pp over Tesseract, 40× smaller than Manga-OCR (11 MB vs 440 MB), and the only engine that recognizes characters across all CJK regions in a single model. Only cnocr beats SCER on raw latency (12.7 ms vs 24.7 ms), and that advantage comes at a 31.6 pp accuracy deficit.

Quantization and acceleration are bracketed with explicit accuracy-preservation gates. Numerical parity between the PyTorch and Keras graphs (8 random batches, 5 output heads) holds at max-abs-diff under 1e-5 on every head:

Output head	max-abs diff	mean-abs diff	tolerance gate
embedding (L2-norm)	2.83e-07	6.22e-08	1e-5
radical (logits)	2.38e-06	2.40e-07	1e-5
total_strokes	1.91e-06	4.77e-07	1e-5
residual_strokes	1.85e-06	4.21e-07	1e-5
ids_top_idc	2.05e-06	3.18e-07	1e-5

INT8 quantization parity (1000-sample held-out validation pack) shows zero accuracy loss with full prediction agreement:

Metric	PT FP32	TFLite INT8	Δ
top-1 accuracy	96.90 %	96.90 %	0.00 pp
top-5 accuracy	97.70 %	97.70 %	0.00 pp
PT ↔ TFLite prediction agreement	—	—	100.0 %
sample-wise embedding cosine sim	—	—	mean 0.9978 / min 0.9646

Coral acceleration vs CPU on the same TFLite model:

Stage	Pi CPU (ai-edge-litert)	Pi + Coral (libedgetpu)	Δ
forward latency	14.65 ms	11.23 ms	−23 %
cosine NN (CPU numpy, both modes)	13.46 ms	13.46 ms	(same)
end-to-end	28.52 ms	24.70 ms	−13 %
1000-pack top-1	96.90 %	96.90 %	0.00 pp ✓
38-img top-1	81.6 %	81.6 %	0.00 pp ✓

The Coral cache split is the empirical justification for the architectural pivot. The Coral USB Accelerator is a memory-bandwidth-bound device: mapping ops to the TPU is necessary but not sufficient, because the model weights must also fit on-chip. The baseline maps cleanly with 36 of 36 ops but pays no acceleration because 87 % of weights stream over USB; SCER maps cleanly with 41 of 41 ops and obtains a real 13 % end-to-end latency reduction because 69 % of weights are cached on-chip.

Model	Total size	On-chip cache	Off-chip stream	Coral speedup
Baseline (98K-FC INT8)	59 MB	7.59 MB (13 %)	51.43 MB (87 %)	≈ 0 % (memory-bound)
SCER INT8	11 MB	7.58 MB (69 %)	3.41 MB (31 %)	−13 % vs CPU

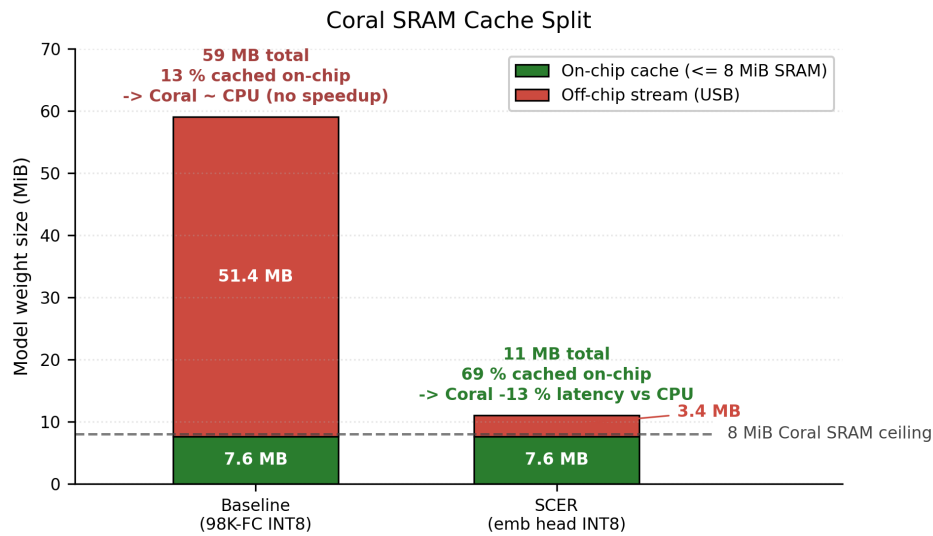


Figure 5. Coral USB Edge TPU on-chip vs off-chip weight split. After `edgetpu_compiler -s`

Both the baseline and SCER compile with 100 % op coverage. The Coral SRAM caps at 8 MiB (dashed line). The baseline's 50 MB FC head pushes 87 % of weights off-chip onto the USB streaming path, which hides the matrix-multiply acceleration behind PCIe latency. SCER's smaller graph fits 69 % on-chip and yields a real 13 % end-to-end latency reduction over the same INT8 model on Pi CPU.

A surprising emergent dynamic was observed during ArcFace training. The embedding top-1 on a held-out 12K validation set jumps from 37.5 % at epoch 10 to 78.3 % at epoch 11, a $2.1\times$ improvement in a single epoch when the cosine LR resets at the $m=0.5$ curriculum boundary. A second wave across epochs 17 $\rightarrow$ 20 adds another +12 pp on the cosine-LR tail, reaching 95.2 % top-1. Four diagnostics characterize the resulting embedding space: 100K random anchor pairs have mean cosine +0.0004 ($\sigma = 0.119$, max 0.735), confirming no anchor collapse on the 128-d hypersphere; the structure-filter-only path reaches 80.3 % on the 12K val while the embedding alone reaches 95.0 %, so the structure heads currently act as a latency optimization rather than an accuracy improvement; the 1000-pack PT FP32 reaches 96.90 % (+62.60 pp over the epoch-10 baseline of 34.30 %); and the 38-image real-world test reaches 81.6 % top-1 / 92.1 % top-5.

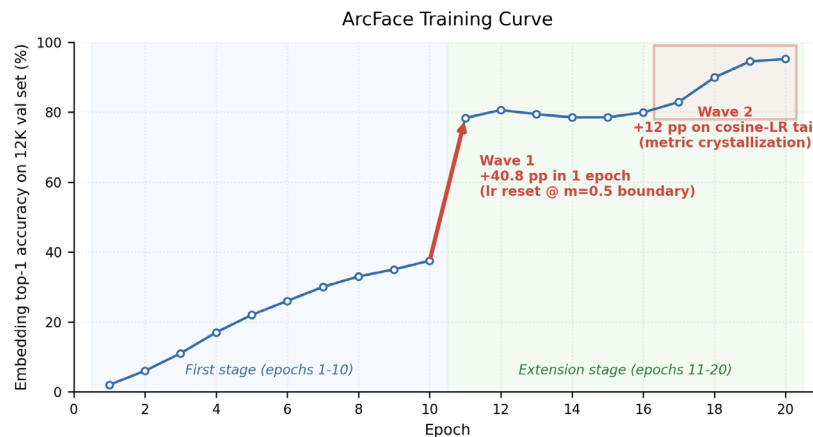


Figure 6. Two-wave cluster crystallization in ArcFace training.

Embedding top-1 on the 12K-sample held-out validation set is plotted across all 20 epochs of the SCER training run. The first stage (blue panel, epochs 1 to 10) ends at 37.5 % top-1 with a smooth-but-modest improvement curve. The extension stage (green panel, epochs 11 to 20) shows two non-linear emergent jumps. Wave 1 adds

+40.8 pp in a single epoch when the cosine LR resets at the $m=0.5$ curriculum boundary. Wave 2 adds +12 pp on the cosine-LR tail as the metric space crystallizes. Final top-1 is 95.2 %.

The current end-to-end latency on the Pi + Coral platform is 24.7 ms / character. The 13.5 ms cosine NN on CPU is the only off-TPU compute on the path and the dominant bottleneck.

Stage	Hardware	Latency	TPU?
Pi Camera capture + preprocess (live demo only)	CPU	0.5 ms	–
TFLite forward (ResNet-18 + 5 heads + L2-norm)	Coral Edge TPU	11.2 ms	✓
INT8 → FP32 dequantize + L2 re-norm	CPU	0.1 ms	–
Cosine NN: emb @ anchors.T, (1, 128) × (128, 98169)	Pi CPU (numpy)	13.5 ms	✗
argsort top-5	CPU	< 0.1 ms	–
End-to-end	mixed	24.7 ms	45 % Coral / 55 % CPU

Three structural reasons currently prevent the cosine from running on the Coral: the 47 MB FP32 anchor matrix (12 MB at INT8) does not fit the 8 MiB on-chip SRAM, the matrix is runtime data rather than a fixed compile-time weight (and edgetpu_compiler only caches fixed weights), and folding it into the model graph as an FC weight would lose the open-set property. Three independent pure-latency remediation paths are nevertheless available without redesign:

Path	Mechanism	Projected total latency	Open-set?
A. Approximate NN index (HNSW / FAISS) on CPU	Replace exhaustive matmul with a logarithmic graph search, achieving roughly 1.5 ms cosine	~13 ms	preserved
B. Pre-filter via the existing structure heads (98K → ~50 candidates)	Match predicted radical (top-5) $\cap$ stroke count (± 1) before cosine, with sub-millisecond cosine on the filtered set	~12 ms	preserved
C. Anchor-DB chunking + on-Coral matmul	Split anchors into 10×4.7 MB chunks, run a partial FC per chunk on-Coral, then accumulate top-K	~6–8 ms (estimate)	preserved
(Combined A + B)	ANN over a structure-filtered candidate set	single-digit ms (≤ 8)	preserved

The current 24.7 ms is real-time on the Pi 5 and is already $32\times$ faster than Manga-OCR (788 ms) at higher accuracy. Single-digit-ms operation is reachable through the optimization paths above; they were not implemented within the lab's time budget because they constitute pure speed optimizations on top of an already working accuracy baseline.

Hardware-incompatible directions (Capsule Net with dynamic routing, Mixture of Experts with conditional gating, stroke-decomposition VAE) were excluded due to Coral op-set constraints. However, if I had another chance and time to initiate custom DNN project, I would definitely try anchor-search acceleration (paths A through C above, 1–4 weeks per path with current accuracy preserved), self-supervised backbone pre-training (DINOv2 / MAE on unlabeled CJK image corpora, the one Coral-compatible architecturally novel direction not pursued, which could reduce the synthetic corpus size and improve generalization to photographic input), stronger structure heads (current filter recall 81 % vs embedding top-1 of 95 %, which would unlock path B), and end-to-end IoT extensions (battery-powered handheld operation, multi-character page capture and segmentation, on-device anchor-DB updates triggered by a user gesture).

5. Conclusion

SCER delivers a universal CJK character recognizer that is Pareto-dominant over every tested commodity engine across size, latency, accuracy, and CJK coverage on a Raspberry Pi 5 + Coral USB stick. The

architecture survives INT8 quantization and Coral hardware acceleration at zero accuracy loss, with the full quantitative evidence presented in the tables above. The contribution is the joint algorithm-and-acceleration co-design that Track 2 specifies as the deliverable: the same deploy chain produces a complete edge IoT system on the standard Lab 3 kit, exercised end to end during the live camera demo with no cloud round-trip. The dominant remaining bottleneck is the cosine NN search on the Pi CPU; three remediation paths and their projected latencies are detailed in the table above. The one architecturally novel direction consistent with the Coral envelope and not pursued in this work is self-supervised backbone pre-training (DINOv2 or MAE on unlabeled CJK corpora), which would improve generalization to photographic input.

6. References

- [1] J. Deng, J. Guo, N. Xue, S. Zafeiriou. “**ArcFace: Additive Angular Margin Loss for Deep Face Recognition.**” *CVPR 2019*. <https://arxiv.org/abs/1801.07698>
- [2] K. He, X. Zhang, S. Ren, J. Sun. “**Deep Residual Learning for Image Recognition.**” *CVPR 2016*. <https://arxiv.org/abs/1512.03385> (ResNet-18 backbone, torchvision implementation, weights trained from scratch on the synthetic CJK corpus.)
- [3] JaidevAI. **EasyOCR**. <https://github.com/JaidevAI/EasyOCR> (CRNN + CRAFT detector. Per-language Reader instances ja, ch_tra, ch_sim benchmarked.)
- [4] Google. **Tesseract OCR 5.5**. <https://github.com/tesseract-ocr/tesseract> (PSM 10 single-character mode with jpn+chi_tra+chi_sim traineddata.)
- [5] breezedeeus. **cnocr 2.3**. <https://github.com/breezedeeus/cnocr> (PP-OCRv5 ONNX weights via the RapidOCR backend.)
- [6] kha-white. **Manga-OCR (kha-white/manga-ocr-base)**. <https://huggingface.co/kha-white/manga-ocr-base> (ViT encoder + GPT-2 decoder, trained on Japanese manga panels.)
- [7] PaddlePaddle / Baidu. **PaddleOCR 3.5 / PP-OCRv5**. <https://github.com/PaddlePaddle/PaddleOCR> (Adapter is implemented but the underlying paddlepaddle 3.x SIGSEGVs on ARM64 + Python 3.13 in SaveOrLoadPirParameters. Graceful skip with a clear runtime error.)
- [8] Google Cloud. **Cloud Vision API: TEXT_DETECTION**. <https://cloud.google.com/vision> (Adapter implemented. Not enabled in the default benchmark group due to the GCP billing-account requirement.)
- [9] Google. **Coral USB Accelerator + Edge TPU Compiler v16.0**. <https://coral.ai> (Hardware accelerator and the toolchain that compiles INT8 TFLite to Edge TPU bytecode.)
- [10] Google AI Edge. **ai-edge-litert 2.1.3**. <https://github.com/google-ai-edge/LiteRT> (TFLite runtime successor. The only TFLite Python interpreter with a Python 3.13 wheel for ARM64 at the time of this lab.)
- [11] Unicode Consortium. **UniHan database**. <https://www.unicode.org/charts/unihan.html> (Source of CJK codepoint coverage and radical / stroke metadata.)
- [12] KanjiVG project. **Kanji stroke decomposition**. <https://kanjivg.tagaini.net> (Stroke and radical labels for Japanese kanji.)
- [13] e-hanja. **Korean hanja taxonomy**. <https://hanja.dict.naver.com> (Korean hanja stroke order and radical labels.)

[14] M. Wikner. **MakeMeAHanzi**. <https://github.com/skishore/makemeahanzi> (Chinese stroke graphics, CC BY 4.0.)

[15] Y. Lee. **Sinograph Explorer (this project)**. <https://github.com/Yoonkyu-Lee/sinograph-explorer> (Repository containing all training, deploy, and demo code referenced in this report.)